

Tiny TPU: From RTL to PnR

Minh Nguyen | Manh Tran
University of Massachusetts Amherst

Introduction

- The computational bottle neck of modern Deep Learning is not only the execution of matrix multiplication, but it is also moving to the significant latency and power overhead that comes from fetching weights and activations from SRAM.
- This project presents the end-to-end design, verification, and physical implementation of a custom Tensor Processing Unit (TPU) specifically architected to mitigate this memory bandwidth limitation

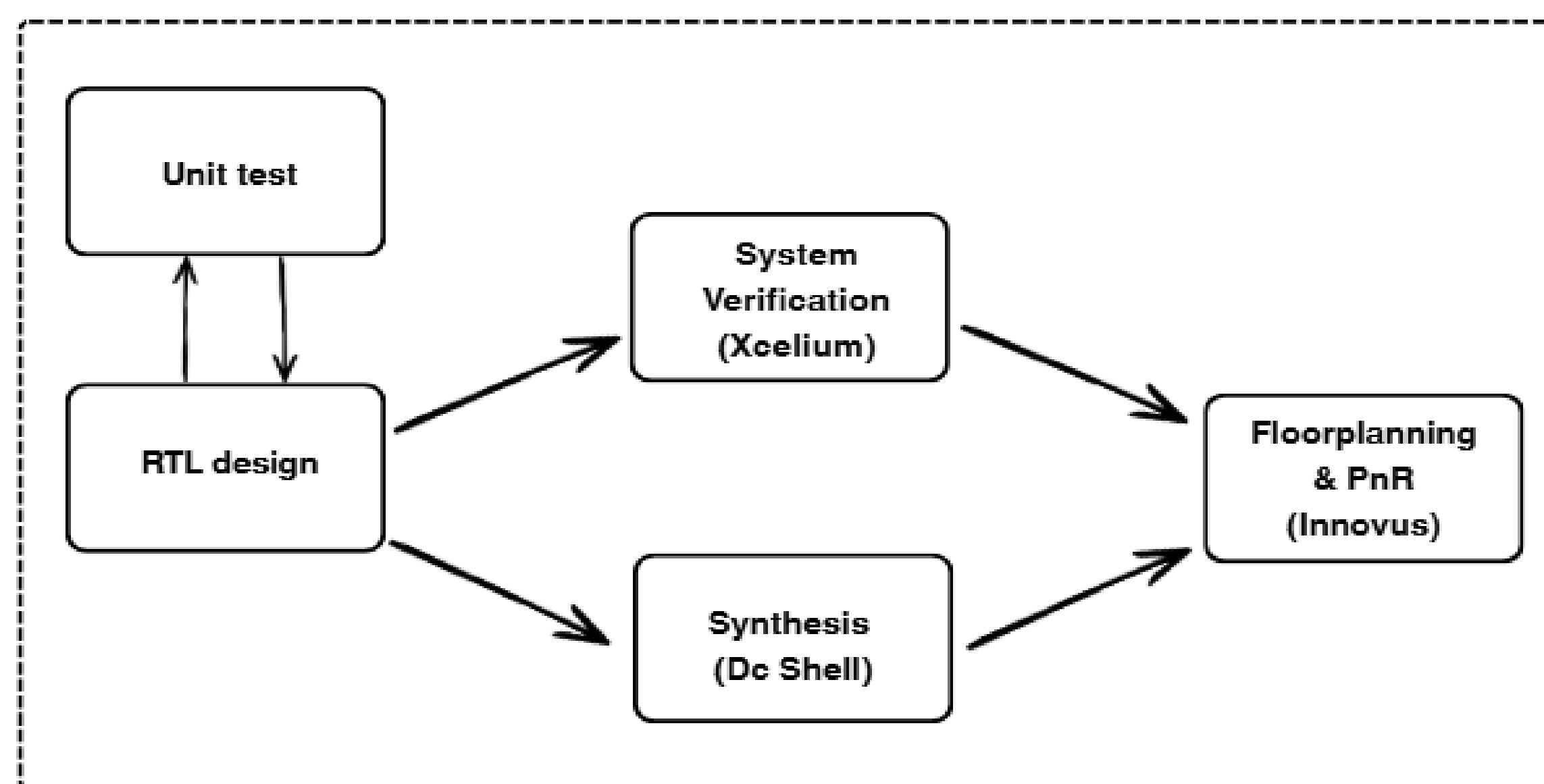


Fig 1: Project Design Flow

Project Scope

- Architecture:** Designed a 16x16 Weight-Stationary Systolic Array from scratch in SystemVerilog to maximize data reuse
- Verification:** We implemented a Universal Verification Methodology (UVM) testbench and perform unit testing on each modules to subject the architecture to functional test, continuous volume stress tests, and mid-flight asynchronous resets
- Physical Implementation:** Successfully pushed the verified RTL through Logic Synthesis and Place & Route (PnR) targeting a 45nm process node

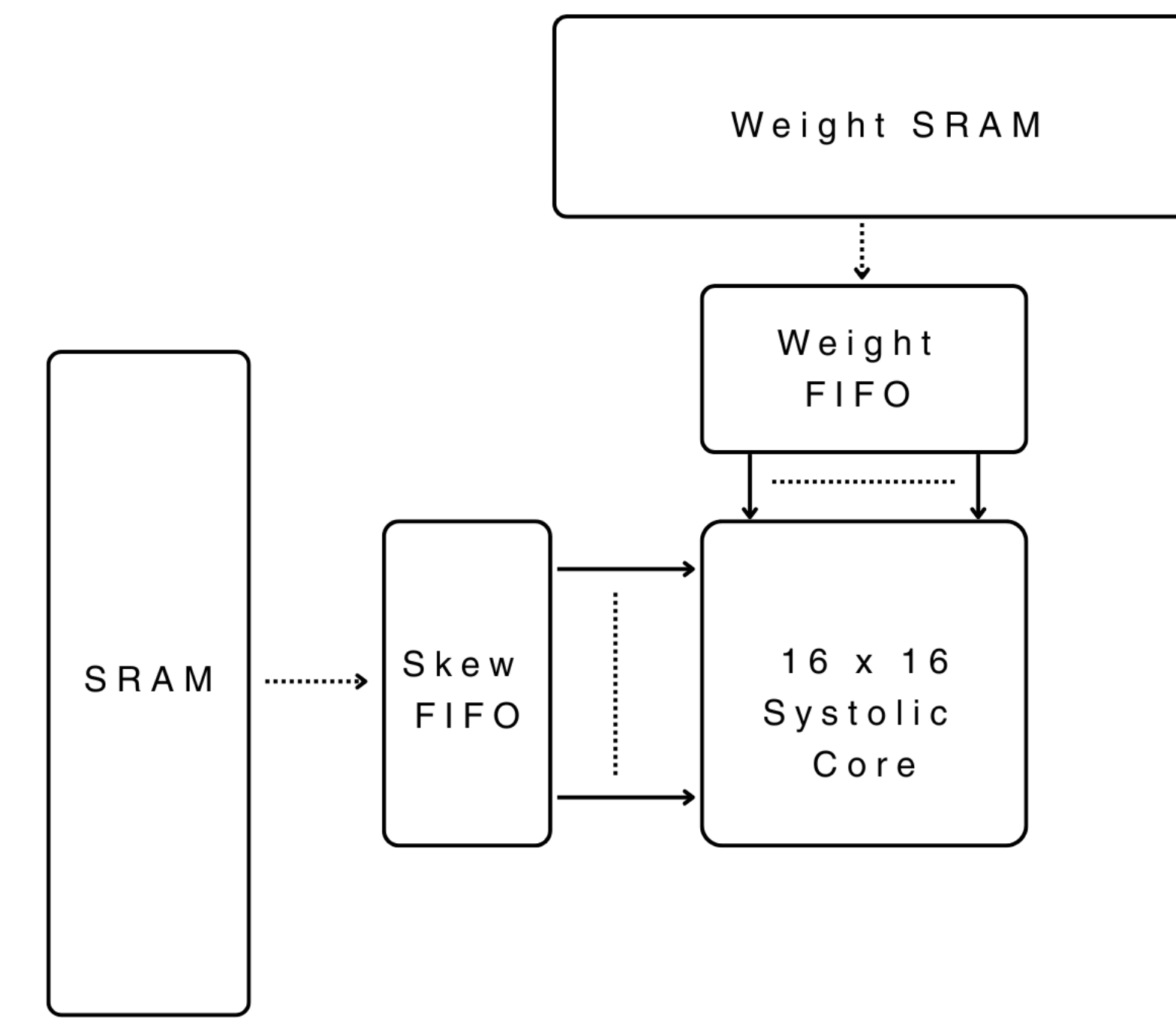


Fig 2: DUT Diagram

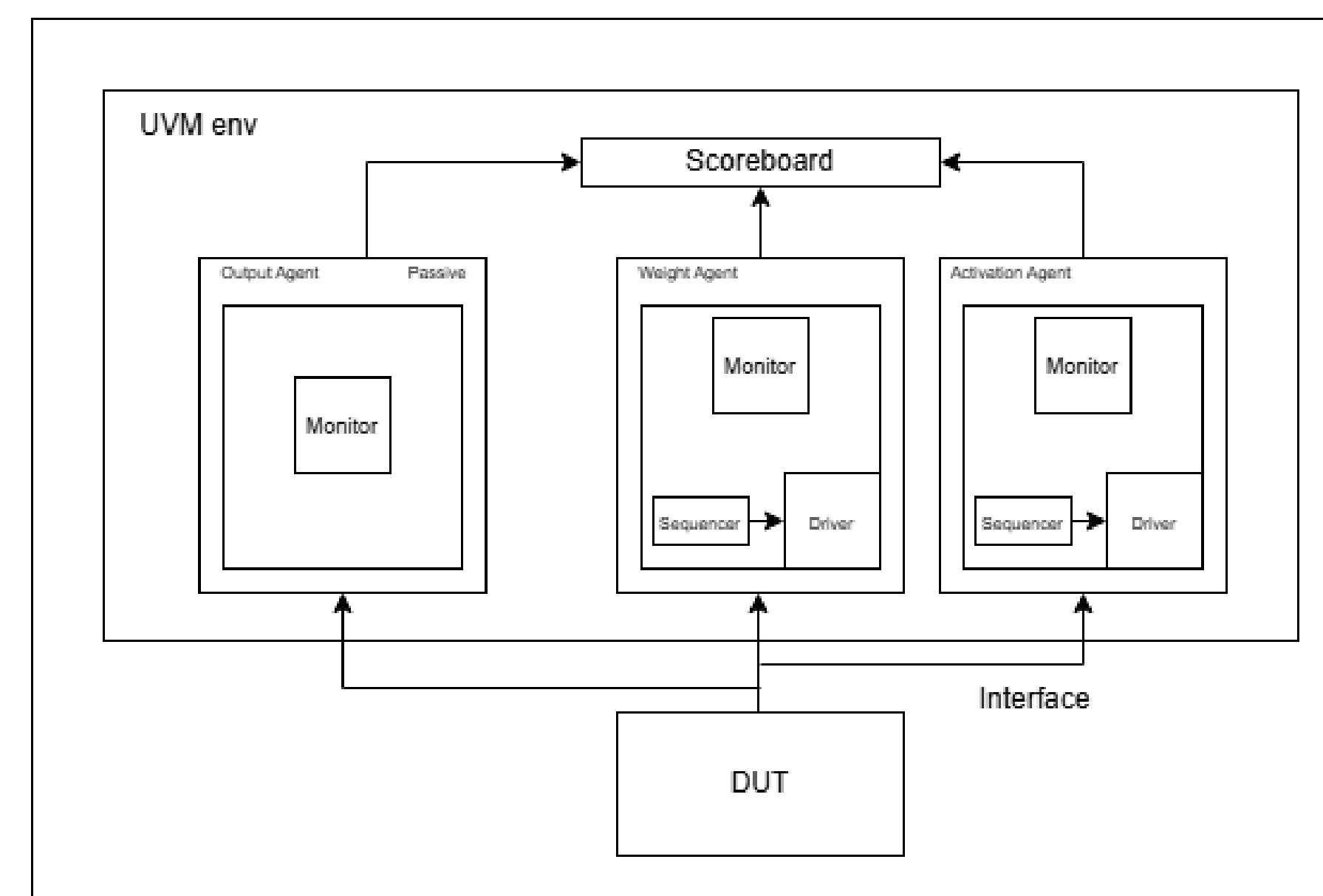


Fig 3: UVM Architecture

Hardware Architecture

The TPU is built around a highly parallel, spatially aware datapath designed to maximize data reuse and achieve PnR execution in 45nm silicon.

- 16x16 Systolic Array:** The computational core relies on a tightly coupled grid of 256 Processing Elements (PEs). It utilizes a **Weight-Stationary** dataflow, pinning weights locally to eliminate redundant memory fetches
- Processing Elements (PE):** Each PE contains a high-speed Multiply-Accumulate (MAC) unit. During execution, activations are passed horizontally, while accumulating partial sums cascade vertically.
- Memory Organization:** To feed the array, the design integrates local Activation and Weight SRAMs decoupled by Asynchronous FIFOs.

Verification Environment

To guarantee that the design works before going for synthesizing the design, we have performed 2 methods for testing our design:

- Unit Testing (while developing design):** We test out each modules of the design with Vivado and verifying the functionality through looking at waveforms after simulation simple testbench.
- UVM (after finishing the design):** We implemented an UVM environment and creates testbench for testing edge cases and pulling constrained random inputs for both input and output. A reference model was built into the UVM scoreboard to dynamically calculated the expected outputs and testing the timing of the pipeline and the functionality of the design. (Architecture is shown in Fig 3)

Results

- The design achieved functional correctness against the reference model, with inputs for weights and activation from different cases (negative, positive and 0). The design was able to perform correctly under processing 100 continuous, randomized matrix streams without pipeline stalls and demonstrated flawless recovery from asynchronous mid-flight resets.
- The verified RTL was synthesized using Synopsys Design Compiler. The gate-level netlist successfully met strict setup timing constraints for a **200 MHz** (5.0 ns) target clock, achieving a healthy positive slack of **+2.71 ns**.

Achieved Frequency	Timing Slack	Total Area	Dynamic Pow	Static Pow
~436 MHz	+2.71 ns	1.92 mm2	95.8 mW	0.12 mW

Table 1: Result after synthesis